Embedded Data Structure
(Lecture Note 19)

Searching Algorithms (2)

2025. 5. 22

Prof. Wonjun Kim

School of Electrical and Electronics Engineering

Review of the Last Class

- **Find a specific key value in your data structure**
  - **Searching algorithm : init. + insert + delete + search**
  - **Remember the scheme of the binary search**
    - Based on the sorted data (by any type of sorting algorithms)
    - Keep the sorted data structure for insert and delete operations
    - Complexity : O(logN)

    ※ Performance can be improved via interpolation technique
  - **Today's topic : sequential search (baseline of search algorithms)**

Sequential Search (1/12)

- **Simple search with unsorted data**
  - **It compares data N/2 times averagely (how about worst case ?)**

How many time do we need to compare ? ➡ For '**W**', how many times ?

Pseudo code :

```
1. i = 0
2. while(array[i] != key && i < # of data)
     1. i++;
3. If(i < # of data) return i; else return -1;
```

➡ Return index (not value)

Sequential Search (2/12)

- **Insert and delete**
  - **Insert at the end of the data structure**

T O L E A R N S E A R C H I N G A L G O R I T H M **Z**

: Number of data is changed as 22 → 23

  - **Delete a specific data and fill the corresponding space**

T O L E A R N S **E** A R C H I N G A L G O R I T H M Z

T O L E A R N S A R C H I N G A L G O R I T H M Z

T O L E A R N S A R C H I N G A L G O R I T H M Z

: Number of data is changed as 23 → 22 ※ Lazy deletion

Sequential Search (3/12)

▪ C implementation

- **Three main functions : search, insert, and delete**

```
int seq_search(int key, int *a, int *num)
{
    int i = 0;
    while(a[i] != key && i < *num) i++;

    return (i < *num ? i : -1);
}
```

Why do we use a pointer ?

Return the index

```
int seq_insert(int key, int *a, int *num)
{
    a[(*num)++] = key;

    return [          ] ;
}
```

Return the index of inserted key

Sequential Search (4/12)

KU KONKUK UNIVERSITY

▪ C implementation – cont'd

- **Delete function : delete and copy**

```
int seq_delete(int key, int *a, int *num)
{
    int p, i;
    if(*num > 0)
    {
        if((p = seq_search(key, a, num)) < 0) return -1;
        for(i = p+1; i<*num; i++)
            a[i-1] = a[i];
        (*num)--;
        return p;
    }
    return -1;
}
```

} **Change this part using memcpy()**

Memory management (why ?)

```
int data[SIZE];
int nitem;
...
if(nitem < SIZE) seq_insert( [          ] );
else printf("Memory is full !!₩n");
```

Sequential Search (5/12)

▪ Improvement with hit rates

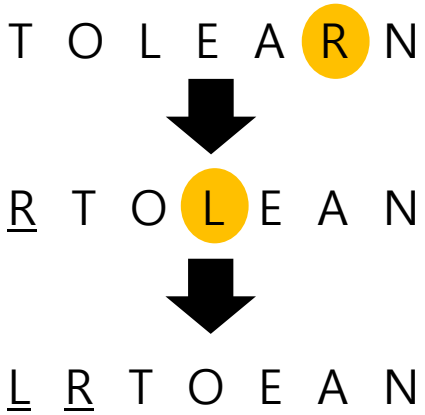
- **Procedure (1) : place a value at first according to the hit rate**

  To do this, we need to have training time to estimate counts

- **Procedure (2) : place a value searched previously at first**

```
int seq_f_search(int key, int *a, int *num)
{
    int p, i;
    if((p=seq_search(key, a, num)) < 0)
        return -1;
    for(i = p-1; i>=0; i--)
        a[i+1] = a[i];
    a[0] = key;

    return 0;
}
```



< Self organization >

Sequential Search (6/12)

KU KONKUK UNIVERSITY

▪ Generalization of sequential search algorithm

- **Using ① void pointer and ② function pointer**

Sort only for "int"	Sort for all the types
Store key : int min	void *min; min = malloc(width);
Array elem : a[y]	*(type)((char *)base + y*width)
Comparison : min > a[x]	fcmp(min, (char *)base + x*width) > 0
Assign : a[y] = min	memcpy((char *)base + y*width, min, width)

※ Is it possible to conduct pointer operations for "void" type ?

```
int intcmp(const void *a, const void *b)
{
    return (*(int *)a - *(int *)b);
}
```

What does it mean ?

Change the type what you want !

Sequential Search (7/12)

- **Generalization of sequential search algorithm – cont'd**
  - **Using of void pointers and function pointers**

```
#include <stdio.h>
#include <stdlib.h>
#include <windows.h>

typedef int (*FCMP)(const void *, const void *);
```

➔ What does it mean ?

```
int lv_search(void *key, void *base, size_t *num, size_t width, FCMP fcmp);
int lv_insert(void *key, void *base, size_t *num, size_t width, FCMP fcmp);
int lv_delete(void *key, void *base, size_t *num, size_t width, FCMP fcmp);
int lfv_search(void *key, void *base, size_t *num, size_t width, FCMP fcmp);
```

You need to define a variable using size_t (e.g., size_t nitem)

Sequential Search (8/12)

- **Generalization of sequential search algorithm – cont'd**
  - **Four functions need to be converted**

```
#define BASE(i)   ((char *)base + (i)*width)

int lv_search(void *key, void *base, size_t *num, size_t width, FCMP fcmp)
{
   int i = 0;
   while(fcmp(BASE(i), key) != 0 && i < *num) i++;
   return (i < *num ? i : -1);
}

int lv_insert(void *key, void *base, size_t *num, size_t width, FCMP fcmp)
{
   memcpy(BASE((*num++)), key, width);
   return *num-1;
}
```

Sequential Search (9/12)

- **Generalization of sequential search algorithm – cont'd**
  - **Four functions need to be converted – cont'd**

```
int lv_delete(void *key, void *base, size_t *num, size_t width, FCMP fcmp)
{
    int p, i;
    if(*num > 0){
        if((p = lv_search(key, base, num, width, fcmp)) < 0)
            return -1;
        for(i = p+1; i < *num; i++){
            memcpy(                                    );
        (*num)--;
        return p;
    }
    return -1;
}
```

Sequential Search (10/12)

- **Generalization of sequential search algorithm – cont'd**
  - **Four functions need to be converted – cont'd**

```
int lfv_search(void *key, void *base, size_t *num, size_t width, FCMP fcmp)
{
    int i = 0, j;
    void *v;
    while(fcmp(BASE(i), key) != 0 && i < *num) i++;
    if(i >= *num) retun -1;
    v = malloc(width);
    memcpy( v, BASE(i), width);
    for(j=i-1; j>=0 ; j--)
        memcpy( [                    ] );
    memcpy( [                    ] );

    return 0;
}
```

Why do we need mem. allocation ?

Sequential Search (11/12)

- **Practical example (1)**
  - **Generate 10,000 random values (range 0~9,999)**

    ➔ Store them data[] and find a specific number

Make your code using four basic functions !
(seq_search, seq_insert, seq_delete, seq_f_search)

KU KONKUK UNIVERSITY

▪ Practical example (2)

- **Find a student ID in your records (with struct)**

: Make some student records and try to find a specific ID

```
#define MAX         100
...
typedef struct _card{
    int no;
    char name[20];
}card;

card table[MAX], key;
int nitem = 0, pos;

int card_cmp(const void *a, const void *b){
    return [                    ]
}
...
```

```
key.no = FIND;
pos = lv_search( [                    ] );
...
printf("No=%d, Name=%s₩n", [          ] );
```

Use generalized functions !

Summary

- **Baseline of searching algorithms**
  - **Sequential search (without sorting)**
  - **Remember the generalized version of each function**